SYSTEM DESIGN DOCUMENT | PS VITA | GODOT 3.5

SLEEZE DISCO
Beat Kune Do

A 2.5D rhythm-action beat 'em up where every punch lands on the beat — rebuilt as a real Vita game: hardware-accelerated, parallax-deep, and neon-lit at native 960 x 544.

Target: 60 FPS combat / 30 FPS floor | 960 x 544 native | Godot 3.5 GLES2 | No PBR | No heavy shaders



Target look — not a mockup promise. Section 4 breaks this exact image into layers the Vita can actually draw: baked-light backgrounds, pre-rendered fighters, additive glow sprites. If a layer cannot hit budget, it does not ship.

What this document is

The complete build spec: engine choice, visual pipeline, game-loop architecture, rhythm and combat systems, performance budgets, and the exporter chain from PC to an installable VPK. A developer should be able to build the game from this document alone.

What changed from v1

The first prototype proved the loop with a CPU software framebuffer — and looked retro because of it. This spec keeps its proven timing and moves, and replaces the renderer, art pipeline, and scene structure entirely. See page 2.

Working title: Sleeze Disco — Beat Kune Do | Original game inspired by the rhythm-fighting idea of Dead as Disco. No Dead as Disco assets, music, characters, or code are used. Dead as Disco itself is an Unreal Engine 5 PC title with no Vita version; this is a native reimagining of the genre for Vita hardware, not a port.

READ THIS FIRST

The first build looked retro for three fixable reasons — none of them is the Vita

The criticism was correct, and the diagnosis matters because it decides the whole rebuild. PSP-era and early homebrew games look the way they do for specific technical reasons. The Vita is dramatically more capable than a PSP — roughly four times the pixels, a quad-core CPU, and a tile-based PowerVR GPU built for textured, lit 2D and light 3D. The prototype used almost none of that.

Cause in v1	Why it read as "retro"	Fix in this spec
CPU software framebuffer	Every pixel drawn by the CPU. No texture filtering, no blending modes worth using, no free scaling or rotation — so art had to stay flat and blocky.	Hardware rendering through Godot GLES2 to the Vita GPU (sceGxm underneath). Filtered textures, additive blending, and tinting become effectively free.
Flat vector-style shapes	No baked light, no rim light, no shadows. Fighters read as icons, not characters in a place.	Pre-rendered sprites: characters are drawn or rendered once at high quality with light, rim, and shadow baked in, then shipped as sprite sheets. The Vita only has to stamp them.
One flat background plane	No depth. Real Vita and late PSP games sell scale with layered parallax even when the gameplay is 2D.	Five parallax layers plus a foreground occluder layer, a reflective floor strip, and animated neon signs (Section 4).

> **The rule that follows:** never draw on the CPU what the GPU can stamp, and never compute at runtime what can be baked at authoring time. Light, shadow, glow falloff, and reflections are baked into art on PC. The Vita spends its budget on movement, timing, and particles — the things the player actually feels.

What carries over from v1 (it was right, keep it)

- **Judgement windows:** Perfect within +/-60 ms pays +50% damage, Good within +/-120 ms pays +10%, off-beat hits still land at base damage and are never punished. Forgiving timing is why the loop feels like a fight instead of a test.
- **The five verbs:** Punch (Square), Kick (Triangle), Drumstick throw (Circle), Dodge (Cross), Counter (L, on a red flash), Takedown (R + Square, full meter).
- **The BPM ladder:** Neon Alley at 100 BPM, Dive Bar at 120 BPM, Idol VEX boss at 140 BPM — plus Infinite Disco and a My Music BPM picker.
- **Enemy readability:** three archetypes the player can tell apart at a glance, and a boss who needs four Perfects to stun.

Everything else — renderer, art, scene layout, audio mixing, save format — is replaced by this document.

1 VISION

You are not fighting near the music. You are fighting in it.

One-sentence pitch: a fallen singer fights through his old band's neon venues, and the crowd, the lights, the enemies, and his own fists all move on the same beat — play well and a brawl turns into a choreographed music video; play badly and it is still a brawl.

1
Fight on the beat

Timing is damage, not a grade at the end. Perfect hits hit harder, build meter faster, and trigger the crowd. The beat is the combat system.

2
Look like a Vita game

Native resolution, filtered pre-rendered sprites, five layers of parallax, baked neon light. If a screenshot could be mistaken for a PSP game, the stage is not finished.

3
Never drop the beat

Frame rate may dip; the audio clock may not. Every performance decision in this document protects beat accuracy first, effects second.

4
Five minutes or fifty

One song is one stage (3-4 minutes). Infinite Disco and score chase give the game a second life after the three-stage story.

Scope — deliberately tight

In scope (v1 ship)	Out of scope (cut on purpose)
3 story stages + boss, Infinite Disco, My Music (BPM picker with bundled tracks), local high scores, calibration screen	Multiplayer, online leaderboards, custom MP3 import on device, character creator, branching story
1 playable fighter, 3 enemy archetypes + palette/pattern variants, 1 three-phase boss	Multiple playable characters, weapons loadouts, RPG skill trees
6 bundled synth/funk tracks with layered stems	Licensed music of any kind

Custom song import is the most-requested feature this genre gets and the first one to sink a Vita schedule — decoding arbitrary files, beat detection, and rights issues all land at once. The BPM picker over bundled tracks delivers 80% of the fantasy at 5% of the risk.

2 ENGINE DECISION

Godot 3.5, GLES2, GDScript — chosen for the exporter that exists and the 2D tools that matter

The request allowed Godot 3.5 or GameMaker Studio. Both have community Vita paths. The decision below is not about which engine is "better" in general — it is about which one gets this specific game onto a Vita with the least risk.

Criterion	Godot 3.5 (chosen)	GameMaker Studio 2 (fallback)
Renderer	GLES2 backend maps cleanly to Vita GPU; no Vulkan requirement	Lightweight 2D runner, also viable
2D tooling	AnimationPlayer, AnimatedSprite, Y-sort, CanvasLayers, particles — all native concepts this design uses directly	Strong sprite tools; timeline/animation control is coarser
Language	GDScript for gameplay; optional C# is **not** used — Mono adds runtime weight and exporter risk on Vita	GML, fast to iterate
Cost / licensing	Free and open source; no export fees	Paid export licensing has changed repeatedly; verify current terms before committing
Vita path	Community Godot 3.x Vita port / exporter targeting sceGxm; treat as a pinned, tested dependency (Section 11)	Community GMS Vita runners exist but are less actively maintained
Team fit	Scenes and resources are text-diffable; the whole game can live in version control cleanly	Project format is less diff-friendly

> **Hard engine rules:** Godot **3.5.x only** (not Godot 4 — its Vulkan-first renderer is the wrong weight class for Vita). Rendering backend **GLES2**. Physics engine: built-in 2D with a fixed 60 Hz tick. No dynamic lights (Light2D) in gameplay scenes, no screen-space shaders beyond one cheap palette/flash effect, no 3D nodes except a fixed camera backdrop quad if needed. Every one of those bans is a frame-rate decision, not a taste decision.

If the Godot Vita exporter blocks

The fallback is not a rewrite: all art, audio stems, beat maps, and tuning data are engine-neutral files (PNG atlases, OGG stems, JSON beat maps). The GameMaker fallback re-implements only the runtime (Sections 6-8) against the same data. That portability is designed in from day one — data first, engine second.

The Vita budget: generous for baked 2D, unforgiving of runtime cleverness

Subsystem	Reality	What this design does with it
Display	960 x 544 OLED, 5 inch	Render at native res, never upscale. Art authored at 1x for backgrounds, 2x for hero fighter (downscaled = crisp).
CPU	Quad-core ARM Cortex-A9, up to 444 MHz in boosted titles	One core for gameplay, audio mixing kept on the audio thread, no per-frame allocations in the hot loop.
GPU	PowerVR SGX543MP4+ (tile-based)	Minimise overdraw and transparent full-screen layers — tile GPUs punish stacked translucency. Max 2 full-screen transparent layers.
Memory	512 MB RAM, 128 MB VRAM (plus shared)	Per-stage VRAM budget 64 MB (Section 9). Atlases streamed per stage, freed between stages.
Storage / load	Memory card, slow random read	One packed resource file per stage; load target under 4 seconds, masked by the track intro count-in.
Input	Full physical controls + touch + rear touch	Physical controls are primary. Touch is menu-only. Rear touch is never required.
Battery / heat	Sustained max clocks heat the device	Menus run at 30 FPS with vsync; gameplay clock policy in Section 9.

> **PSP comparison, for calibration:** a PSP runs at 480 x 272 with a 333 MHz single-issue CPU and 32-64 MB RAM. This design's per-stage texture budget alone exceeds total PSP RAM. There is no hardware excuse for a PSP look — only pipeline excuses, which Section 4 removes.

Frame-rate policy

- **Combat: target 60 FPS, floor 30 FPS.** Simulation is fixed-step (Section 6), so a dip to 30 renders every other simulation step — timing judgements stay identical.
- **Menus, results, calibration: 30 FPS locked.** Saves battery, hides nothing.
- **Cutscenes / stage intros: 30 FPS.** They are camera pans over still art.

4 VISUAL PIPELINE — 2.5D WITHOUT A 3D PIPELINE

Depth is painted and layered, never modelled

2.5D here means a 2D game that borrows depth cues: layered parallax, a floor that reflects, fighters who scale subtly as they move up and down the lane, and foreground pieces that pass in front of the action. No meshes, no PBR, no dynamic lighting pass.

Layer	Contents	Scroll factor	Budget note
L0 Skyline	City silhouette, baked neon signs (static)	0.15x	One 1024 x 512 texture per stage
L1 Far venue	Balconies, crowd silhouettes, speaker stacks	0.35x	Crowd bobs on the beat via a 2-frame swap, not skeletal animation
L2 Mid set	Stage dressing, animated sign strip	0.60x	Animated signs = 4-frame atlas loop, timed to the beat clock
L3 Playfield	Fighters, effects, pickups — Y-sorted	1.00x	The only layer with per-frame logic
L3b Floor	Dance-floor strip with baked reflection gradient	1.00x	Reflection is a flipped, darkened, pre-blurred sprite — not a render target
L4 Foreground	Pillars, hanging lights passing in front	1.30x	Alpha edges only; never covers more than 15% of screen width at once

How fighters get their look

1. **Author big:** draw or render each fighter at 512 px tall with full shading, rim light in the stage's accent colour, and a soft contact shadow painted as a separate blob sprite.
2. **Animate economically:** 8-12 frames per move at 12 fps, stepped — stepped animation on ones and twos reads as style in this genre and halves texture memory versus 60 fps frames.
3. **Bake variants:** hurt flash, Perfect-hit glow rim, and boss phase tints are pre-tinted atlas pages, swapped — not shader uniforms evaluated per pixel.
4. **Atlas everything:** one 2048 x 2048 atlas per character family, one per stage. Texture swaps per frame: target zero.



Archetype readability test: if you cannot tell Stan, Bruiser, and Charger apart in silhouette alone, the sprite fails review — before any colour is applied.

5 GAME LOOP ARCHITECTURE

The audio clock is the master clock. Everything else takes orders from it.

Most games run on wall-clock time and play music alongside. A rhythm game must invert that: the position inside the song — read from the audio hardware clock — is the only truth. If rendering hitches, the beat does not move. If the CPU spikes, the beat does not move.

```
BOOT
  -> Load save, settings, calibration offset
  -> MENU LOOP (30 FPS)

STAGE LOOP (per song)
  1. Stream stage atlas + stems        [masked by count-in]
  2. Start audio; record dsp_start_time
  3. FIXED-STEP SIMULATION (60 Hz accumulator)
       song_time = AudioServer playback position + offset
       beat      = song_time * BPM / 60
       a. Read input (buffered, timestamped vs song_time)
       b. Judge inputs against beat grid
       c. Update fighters, AI, hitstop, camera shake
       d. Spawn / despawn from beat map + wave director
       e. Update combo, meter, crowd, stem mix targets
  4. RENDER (vsync; may skip if sim is behind)
       Draw L0-L4, HUD pulse = f(beat phase)
  5. On song end / player KO -> RESULTS -> unlock, save
```

Scene tree (Godot)

```
Main (autoload: BeatClock, SaveData, AudioDirector)
+-- Menu / Calibration / Results       [CanvasLayer UI]
+-- Stage (Node2D)
|   +-- ParallaxBackground (L0, L1, L2)
|   +-- Floor (Sprite, L3b)
|   +-- YSort (L3: Player, Enemies, Pickups, FX pool)
|   +-- Foreground (Node2D, L4)
|   +-- Camera2D (locked Y, eased X follow + shake)
|   +-- HUD (CanvasLayer: combo, meter, judgement, song bar)
|   +-- WaveDirector, BeatMap (data nodes, no visuals)
+-- Pause (get_tree().paused, audio bus ducked -12 dB)
```

> **Non-negotiables:** object pools for enemies (max 6 active), hit sparks, and damage numbers — zero allocation during a song. All tuning lives in exported resources / JSON, never hard-coded in scripts. Pause freezes the audio clock reference, so resuming never shifts judgement.

A beat map, a DSP clock, and a calibration screen walk into a bar — only the calibration screen is optional to skip

Beat maps are data, authored offline

Each track ships with a JSON beat map: BPM (constant per track in v1 — no tempo changes), first-beat offset, section markers (verse / chorus / breakdown / boss phase), and authored spawn cues. Constant BPM is a deliberate v1 simplification: it makes the clock a multiplication instead of an integration, and eliminates a whole class of drift bugs.

```
{ "track": "neon_alley", "bpm": 100, "offset_ms": 184,
  "sections": [{"at_beat": 0, "name": "intro"},
               {"at_beat": 32, "name": "verse"},
               {"at_beat": 96, "name": "chorus"}],
  "cues": [{"beat": 32, "spawn": "stan", "side": "L"}, ...] }
```

Judgement

Judgement	Window	Damage	Meter	Feedback
PERFECT	+/- 60 ms	x 1.5	+ 12	Gold rim flash, crowd pop, stem layer adds sparkle
GOOD	+/- 120 ms	x 1.1	+ 6	White flash, normal hit sound pitched to the scale
BEAT (off-beat hit)	any other time	x 1.0	+ 2	Plain hit — the fight continues, the groove does not grow
WHIFF / HIT TAKEN	—	—	combo resets	Record-scratch sting, screen desaturates for 200 ms (pre-baked tint swap)

Latency and calibration

- **Two offsets, stored separately:** a global audio offset (device + wired vs speaker) and an input offset, both set on a tap-to-the-beat calibration screen with a 16-tap average and outlier rejection.
- **Defaults ship sane:** 0 ms on the Vita's own speakers/controls — calibration is a refinement, never a gate to playing.
- **Judgement uses song_time at the moment of the button interrupt**, not the frame that happens to render next. Input is timestamped on read and judged against the beat grid, so a 30 FPS dip cannot steal a Perfect.

Five verbs, one meter, and hitstop doing the heavy lifting

Input	Move	Startup / Active / Recovery (frames @60)	Beat role
Square	Punch — fast jab, chains x3	4 / 3 / 8	Chain hits quantise naturally to 8th notes at stage BPMs
Triangle	Kick — slower, longer reach, launcher on 3rd chain hit	8 / 4 / 14	Quarter-note weight; the Perfect-window showcase move
Circle	Drumstick throw — ranged, 1.2 s cooldown	6 / — / 10	Interrupts Chargers; breaks Bruiser guard at Perfect timing only
Cross	Dodge — i-frames, repositions through enemies	2 / 10 i-frames / 6	Dodging exactly on the beat refunds the cooldown instantly (Groove Dodge)
L (on red flash)	Counter — parry window 150 ms after flash	0 / 9 / 12	Counter on the beat = free Perfect judgement, regardless of phase
R + Square (meter full)	Takedown — cinematic nearest-enemy finisher	—	Triggers a 4-beat slow-mo (0.35x sim, music keeps full speed — the flex)

Feel systems (cheap, decisive)

- **Hitstop:** 45 ms freeze on normal hits, 90 ms on Perfects, 140 ms on Takedown impact. Implemented as a sim-time scale, never by pausing audio.
- **Hit pitch:** impact sounds step up a pentatonic scale with combo count (x8 steps, then octave). Costs nothing; makes a long combo sound composed.
- **Camera:** 2-4 px shake scaled by judgement, never more — this is a rhythm game, readability beats spectacle.
- **Combo meter:** fills only from judged hits (table, Section 6). At 25 / 50 / 100 combo, crowd tier rises and a stem layer fades in (Section 8).
- **Health:** 100 HP. Enemy hits cost 6-14 by archetype. No health pickups mid-song; clearing a section restores 15 HP — the song structure is the difficulty structure.

> **Design guardrail:** a player who ignores the beat entirely should still clear Neon Alley at low rank. A player who lives on the beat should S-rank it. Rhythm is the skill ceiling, never the entry fee — that is the whole commercial bet of the genre.

Three silhouettes teach the whole game; the boss examines you on all of it

Enemy	Read	Behaviour	Counter-play
Stan — the fan	Lanky, lunges on the beat	Approaches, winds up for exactly 1 beat (flash on the off-beat), strikes on the beat	Any chain; the tutorial enemy — his wind-up teaches the beat clock
Charger	Lean, low, fast	Circles for 2 beats, dashes across the lane on beat 3	Drumstick interrupt, or Groove Dodge through the dash
Bruiser	Huge, armoured front	Slow advance, guard up; drops guard for 1 beat after every 4th beat of the bar	Perfect-timed hits or drumstick to break guard; 4 Perfects stun him outright

Idol VEX — boss, 140 BPM, three phases

1. **Phase 1 — The Audition (100% to 66% HP):** summons Stans on section cues, attacks in Stan patterns at boss damage. Teaches: boss is just a bigger reader of the same clock.
2. **Phase 2 — The Remix (66% to 33%):** adds Charger dashes and a call-and-response pattern — VEX flashes a 4-beat sequence, the player must answer with the matching verbs on the same beats to break his guard.
3. **Phase 3 — The Encore (33% to 0%):** lights drop to silhouette + rim only (pre-baked stage variant), spawn rate up, VEX vulnerable only on Perfects. Four Perfects in a row stun him for a full bar — the Takedown window.

Stage	BPM	New idea	Length / unlock
1. Neon Alley	100	Stans; beat, chain, judgement	~3 min / open
2. Dive Bar	120	Chargers + Bruisers; counters, Groove Dodge	~3.5 min / clear Stage 1
3. Idol Stage — VEX	140	Boss, all systems	~4 min / clear Stage 2
Infinite Disco	100 to 160 ramp	Endless waves, score chase	Endless / clear Stage 1
My Music	Picker 80-160	Bundled tracks, any stage skin	Per track / open

Wave direction is beat-map driven, not random: spawns are authored to sections so choruses always feel bigger than verses. A density governor caps active enemies at 6 and queues the rest to the next beat — the cap is a performance rule and a readability rule at once.

The mix is a game system: play well and the song gets better

Layered stems, crossfaded by performance

Stem layer	Always?	Enters when
Drums + bass (the clock)	Yes — never ducks except pause	—
Chords / pads	Yes	—
Lead / sparkle	No	Combo 25+ (Crowd Tier 1)
Backing vocals / crowd	No	Combo 50+ (Tier 2)
Full arrangement + ad-libs	No	Combo 100+ or boss Phase 3 (Tier 3)

- **Format:** OGG Vorbis, 44.1 kHz, all stems the same length and start-aligned, decoded as streams — never fully loaded into RAM. Stem gains move in 100 ms ramps on section or tier changes, quantised to the next beat so the mix itself stays musical.
- **Godot wiring:** AudioStreamPlayer nodes on a Music bus, one per stem, started in the same frame from the recorded DSP start time. SFX on a pooled set of players (max 12 voices) on an SFX bus; hit sounds are short WAV/OGG, preloaded.
- **Ducking:** Takedown and boss stings duck music by -6 dB for under 1 s. Pause ducks -12 dB. The clock stem is measured before ducking — ducking never affects timing maths.
- **Authoring rule:** every bundled track must survive as drums+bass alone, because a struggling player will hear that version for minutes. If the minimal mix is boring, the track is cut.

Performance budgets — hard gates, profiled on a real Vita

Budget	Limit	Measured how
Draw calls (gameplay)	Under 80 / frame	Godot debugger on device build, worst chorus section
Transparent full-screen layers	Max 2	Scene audit
VRAM per stage	Under 64 MB textures resident	Atlas manifest per stage
Active enemies / particles	6 / 120 particles max	Pool sizes in code review
Sim step cost	Under 4 ms on device	Fixed-step profiler, boss Phase 3 stress scene
Stage load	Under 4 s, masked by count-in	Cold start on device, 3 runs
Install size	Under 350 MB total VPK content	Package audit before release

Physical controls first. The HUD pulses so your eyes never have to visit it.

Input map (complete)

Control	Gameplay	Menus
D-pad / Left stick	Move in lane (up/down depth, left/right advance)	Navigate
Square / Triangle / Circle / Cross	Punch / Kick / Throw / Dodge (Section 7)	Confirm context / — / Back / —
L / R (+Square)	Counter / Takedown modifier	Tab left / right
Start	Pause (clock-safe, Section 5)	—
Touch	Nothing — hands stay on buttons	Full menu navigation

HUD layout (960 x 544)

- **Top-left:** HP bar + character portrait. **Top-right:** score + rank pace indicator.
- **Bottom-centre:** the Beat Ring — a ring around the Takedown icon that contracts onto the beat; it is the metronome for players who need a visual anchor, and it can be dimmed or hidden in options.
- **Centre popups only:** judgement text (PERFECT / GOOD) spawns at the hit, floats 400 ms, dies. Combo counter lives at the player's shoulder, not in a corner — the eye is already there.
- **Song progress:** a thin 2 px bar at the very top edge with section ticks. Nothing else competes with the fight.

Options that ship in v1

- Calibration (audio + input offsets), Beat Ring opacity / off, screen shake off, hitstop reduced, flashing reduced (photosensitivity), master / music / SFX volumes, left-handed lane flip (mirrors the stage), subtitles for boss callouts (on by default).
- **Difficulty:** Opening Act (wider windows: Perfect +/-80 ms, Good +/-160 ms, enemy damage x0.7), Headliner (standard, the tables in this document), Encore (Perfect +/-45 ms, Good +/-90 ms, enemy damage x1.3, S-rank gate for unlocks).

Save data

One save file: stage unlocks, best rank and score per stage/track, calibration offsets, options, Infinite Disco best. Written at results screens only — never mid-song — and versioned (`save_version`) so a patch never eats a player's progress. Corrupt-save behaviour: rename to `.bak`, start fresh, tell the player plainly.

11 BUILD PIPELINE — PC TO VPK

Iterate on PC in seconds. Prove it on a Vita every week.

1. **Develop on PC** in Godot 3.5 at a locked 960 x 544 window. PC feel is a draft; the Vita build is the truth.
2. **Content pipeline:** art exported to atlases by a scripted packer (no hand-packed sheets); tracks mixed to aligned stems + a beat map JSON validated by a checker script (BPM, offset, cue beats inside the track length) before it can enter the project.
3. **Export:** pinned Godot 3.5.x + pinned community Vita exporter version, recorded in the repo. Export produces the game package; it is wrapped into a VPK with title ID, LiveArea art (icon, background, startup gate) authored at Vita spec — the bubble is part of looking like a real game, not an afterthought.
4. **Install and test** via VitaShell on a real device. No milestone is signed off from an emulator or a PC build.
5. **Performance gate:** the Section 9 budgets are re-measured on the boss stress scene at every milestone. A budget breach blocks new content until it is fixed — content debt is how Vita projects die at 20 FPS.

> **Exporter risk, stated plainly:** community Vita exporters are unofficial and version-sensitive. Mitigations: pin every tool version, keep a known-good export machine image, run the fallback test (Section 2) at Milestone 1 — if Godot-to-Vita fails there, the project pivots to the GameMaker runtime while art, stems, and beat maps are still cheap to carry over. Discovering exporter failure at Milestone 4 is the only unforgivable version of this risk.

Risks and mitigations

Risk	Mitigation
Audio drift over a 4-minute track	Clock read from hardware playback position every step; drift test = full-track judgement log on device at M2
Overdraw collapse in chorus sections	2-layer transparency cap; stress scene profiles the worst authored moment, not an average one
Sprite memory creep	Atlas manifest gate (64 MB/stage); new art cannot merge without a manifest diff
Scope creep toward "real Dead as Disco"	Out-of-scope list on page 3 is contractual for v1; ideas go to a v2 parking lot document

Five milestones, each ending on a Vita — not on a promise

Milestone	Deliverable	Acceptance test (on device)
M1 — The Clock	BeatClock, calibration screen, one test track, judgement logging	Full-track drift under 10 ms; exporter fallback test passed
M2 — The Fight	Player + Stan, full verb set, hitstop, combo, Neon Alley greybox	60 FPS in a 6-Stan stress wave; a non-rhythm player can clear it
M3 — The Look	Final Neon Alley art, 5-layer parallax, stem mixing, HUD	Screenshot test: reads as a Vita game beside commercial 2D Vita titles; budgets green
M4 — The Game	All 3 stages, VEX boss, Infinite Disco, My Music, save, results, ranks	Full playthrough on device, no soft-locks; load and install budgets green
M5 — The Ship	Options/accessibility pass, LiveArea polish, final VPK	Fresh-install test on a second Vita; save-corruption test; 1-hour soak with no crash

Definition of done for this design

- Native 960 x 544, 60 FPS combat with a 30 FPS floor that never changes a judgement.
- A screenshot a Vita owner reads as a real Vita game — parallax, baked light, rim-lit fighters — with zero PBR, zero dynamic lights, zero heavy shaders.
- The forgiving Beat Kune Do loop from v1, intact: Perfect +/-60 ms, Good +/-120 ms, off-beat hits always land.
- An installable VPK, tested on hardware, with a save system that survives patches.

> **First build order, if you start tomorrow:** M1 only. Do not draw a single final sprite until the clock, calibration, and exporter are proven on the device. The first prototype's mistake was not ambition — it was spending its effort on content before the pipeline that decides how the game looks and feels. This order fixes that.

Document version 1.0 — Sleeze Disco: Beat Kune Do. Engine: Godot 3.5.x, GLES2, GDScript. Platform: PlayStation Vita homebrew (VPK). All game names, characters, and tracks in this document are original. Dead as Disco is referenced as genre inspiration only.